

Programmering i C/C++

Sammanfattning
Vad vi gått igenom

Föreläsningar

C

- 1: C-grunder, pekare
- 2: Strängar, programkonstruktioner, strukturer, lagringsklasser
- 3: Pekare till funktioner, libc, filhantering, nätverk
- 4: Dynamisk minnesallokering, preprocessering, kompilering, länkning

C++

- 5: C++: Objektorientering, konstanter, namnrymder, referenser,
- 6: klasser, ärvning, konstruktorer, destruktorer, polymorfism
- 7: Operatoröverladdning, exceptioner, bibliotek (iostream), templater (generell programmering)

GUI

- 8: GUI: Programmering: WxWidgets, fönsterprogrammering, programgrundstruktur, fönsterprogrammering, menyer, händelsehanterare
- 9: DC (Device Context), ritande, utskrift, resurser

- 10: COM, .NET, C#

HelloWorld i C

```
/* helloworld.c */  
#include <stdio.h>  
  
int main(void) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

Kompilering och körning

```
% gcc helloworld.c -o helloworld
```

```
% ./helloworld
```

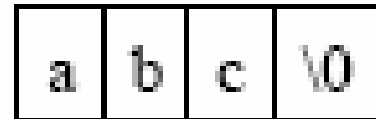
C grunder

- Liknar till sin struktur Pascal eller Fortran (proceduralt språk)
- Värderna sparas i variabler
- Program struktureras genom att definieras och kallas på funktioner
- Programflöden kontrolleras via loopar, if-statements och funktionsanrop
- Användning av pekare (eller indirekt adressering) är centralt

Strängar

- Det finns ingen speciell sträng-typ i C. C är en räkka tecken som avslutas med ett null-tecken

```
char a[] = "abc";  
a[0] = 'a'  
a[1] = 'b'  
a[2] = 'c'  
a[3] = '\0'
```



- I filen `string.h` finns för många användbara sträng-funktioner
– `strlen`, `strcpy`, `strcat`, `strstr`, `strchr`

Synonymer för datatyper

```
typedef unsigned short WORD;
```

```
WORD wIndex; /* samma som unsigned short wIndex */
```

```
typedef struct t_person Person;
```

```
Person jag; /* som struct t_person Jag; */
```

```
typedef struct t_point {  
    WORD x;  
    WORD y;  
} Point;
```

```
Point point;  
point.x = 12; point.y=44;
```

Minneslayout och adresser

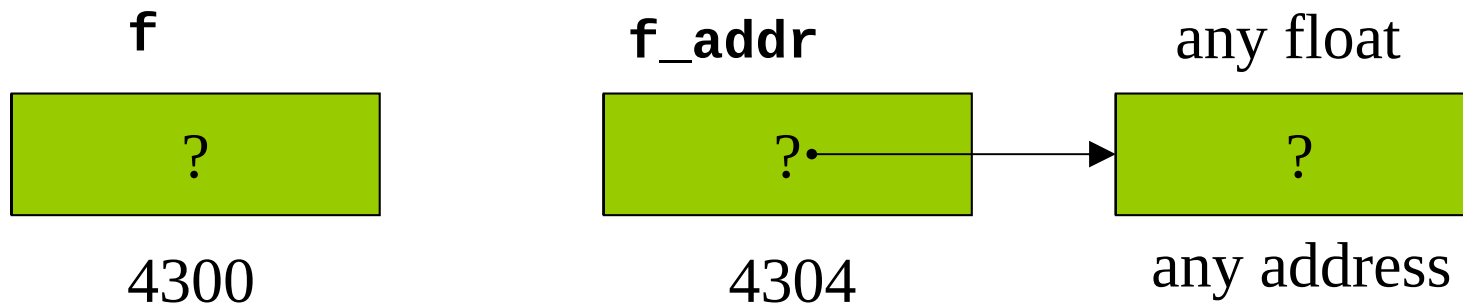
```
int x = 5, y = 10;  
float f = 12.5, g = 9.8;  
char c = 'c', d = 'd';
```

5	10	12.5	9.8	c	d
4300	4304	4308	4312	4316	4317

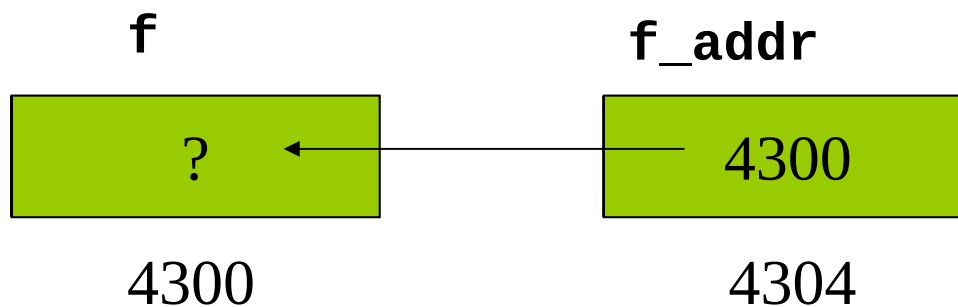
Pekare

- Pekare* = variabel som innehåller adress till annan variabel

```
float f;           /* datavariabel */  
float *f_addr;     /* pekarvariabel */
```

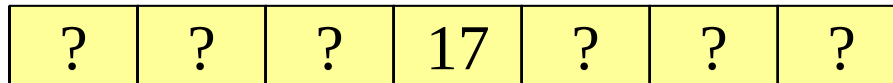


```
f_addr = &f; /* & = adressoperator */
```



Vad är en räkka ?

```
int main(void) {  
    int lottorad[7];  
  
    lottorad[3] = 17;  
}
```



4300

variabeln lottorad är de facto en pekare till det första elementet i räkkan
lottorad[3] kan räknas som
lottorad + 3 * sizeof(int)

Mera pekare

```
int month[12]; /* month is a pointer to base address 430*/
```

```
month[3] = 7; /* month address + 3 * int elements  
=> int at address (430+3*4) is now 7 */
```

```
ptr = month + 2; /* ptr points to month[2],  
=> ptr is now (430+2 * int elements)= 438 */
```

```
ptr[5] = 12;  
/* ptr address + 5 int elements  
=> int at address (434+5*4) is now 12.  
Thus, month[7] is now 12 */
```

```
ptr++; /* ptr <- 438 + 1 * size of int = 442 */  
(ptr + 4)[2] = 12; /* accessing ptr[6] i.e., array[9] */
```

- Nu är month[6], *(month+6), (month+4)[2], ptr[3], *(ptr+3) samma heltalsvariabel !!.

Dynamisk minnesallokering

- Explicit allokering och deallokering

```
#include <stdio.h>
```

```
int main(void) {  
    int *ptr;  
        /* allokera utrymme för ett heltal */  
    ptr = malloc(sizeof(int));  
  
        /* används minnesutrymmet */  
    *ptr=4;  
  
    free(ptr);  
        /* frigör det allokerade minnet */  
}
```

Dynamiskt minne - funktioner

```
#include <malloc.h>
void *malloc(size_t size);
void *calloc(size_t nmemb, size_t size);
void *free(void *ptr);
void *realloc(void *ptr, size_t size);
```

malloc – returnerar pekare till minnesblock av storlek **size**

realloc – ökande / minskade av minne, gemensam det oförändrad

free – frigör minne på pekare **ptr**

Minnesläckor

- Allokerat minne (malloc(), realloc(), calloc()) har inte ett motsvarande free()
- Med tiden växer den allokerade minnesmängden
 - I serverbruk med tiden katastrofalt
 - För klientprogram (typ webbläsare) främst störande
- Hur kan man undvika
 - Allokering / deallokering på välkontrollerade platser
 - Noggrann felhantering (så att inte free() blir bortglömt p.g.a. ovanligt programflöde)
 - Verktyg vid utvecklingsskedet

libc

- [Memory](#): Allocating virtual memory and controlling paging.
- [Character Handling](#): Character testing and conversion functions.
- [String and Array Utilities](#): Utilities for copying and comparing strings and arrays.
- [Character Set Handling](#): Support for extended character sets.
- [Locales](#): The country and language can affect the behavior of library functions.
- [Message Translation](#): How to make the program speak the user's language.
- [Searching and Sorting](#): General searching and sorting functions.
- [Pattern Matching](#): Matching shell ``globs" and regular expressions.
- [I/O Overview](#): Introduction to the I/O facilities.
- [I/O on Streams](#): High-level, portable I/O facilities.
- [Low-Level I/O](#): Low-level, less portable I/O.

Filer - exempel

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    FILE *fp;
    char [] filename = "results.txt";
    fp = fopen(filename, "w");
    if (fp == NULL) {
        printf("Couldn't open %s\n", filename);
        exit(1);
    }
    fprintf(fp, "Hej\n");
    close(fp);
}
```

Formatterad input / output

```
int fprintf(FILE *fp, char *fmt, ...);  
int fscanf(FILE *fp, const char *fmt, ...);
```

- Fungerar som printf and scanf

```
fprintf(fp, "%s %s %u %u\n", cog.first, cog.last,  
    cog.ssn, cog.credit_rating);  
fscanf(fp, "%s %s %u %u\n", &cog.first, &cog.last,  
    &cog.ssn, &cog.credit_rating);
```
- Skillnad på printf fprintf ??
 - Första parameteren i **fprintf** ger filen som man skall skriva till
 - i **printf** är filen som default **stdout**, som finns färdigt deklarerad då prgram startas

Preprocessor – exempel

```
/* test.c */

#define MAX(a,b) ( ((a)>(b))? (a) : (b) )

#ifdef DEBUG
#define DPRINT(x) printf("Debug: '%s\n",
    x);
#else
#define DPRINT(x)
#endif

main () {
    int x = 33;
    int y = 12;

    printf("Max av 2 och 5 ar %i\n",
        MAX(2,5));
    printf("Max av x och y ar %i\n",
        MAX(x,y));

    DPRINT("Nu narmar sig slutet");
}
```

gcc -E test.c -DDEBUG

gcc -E test.c

```
main () {
    int x = 33;
    int y = 12;

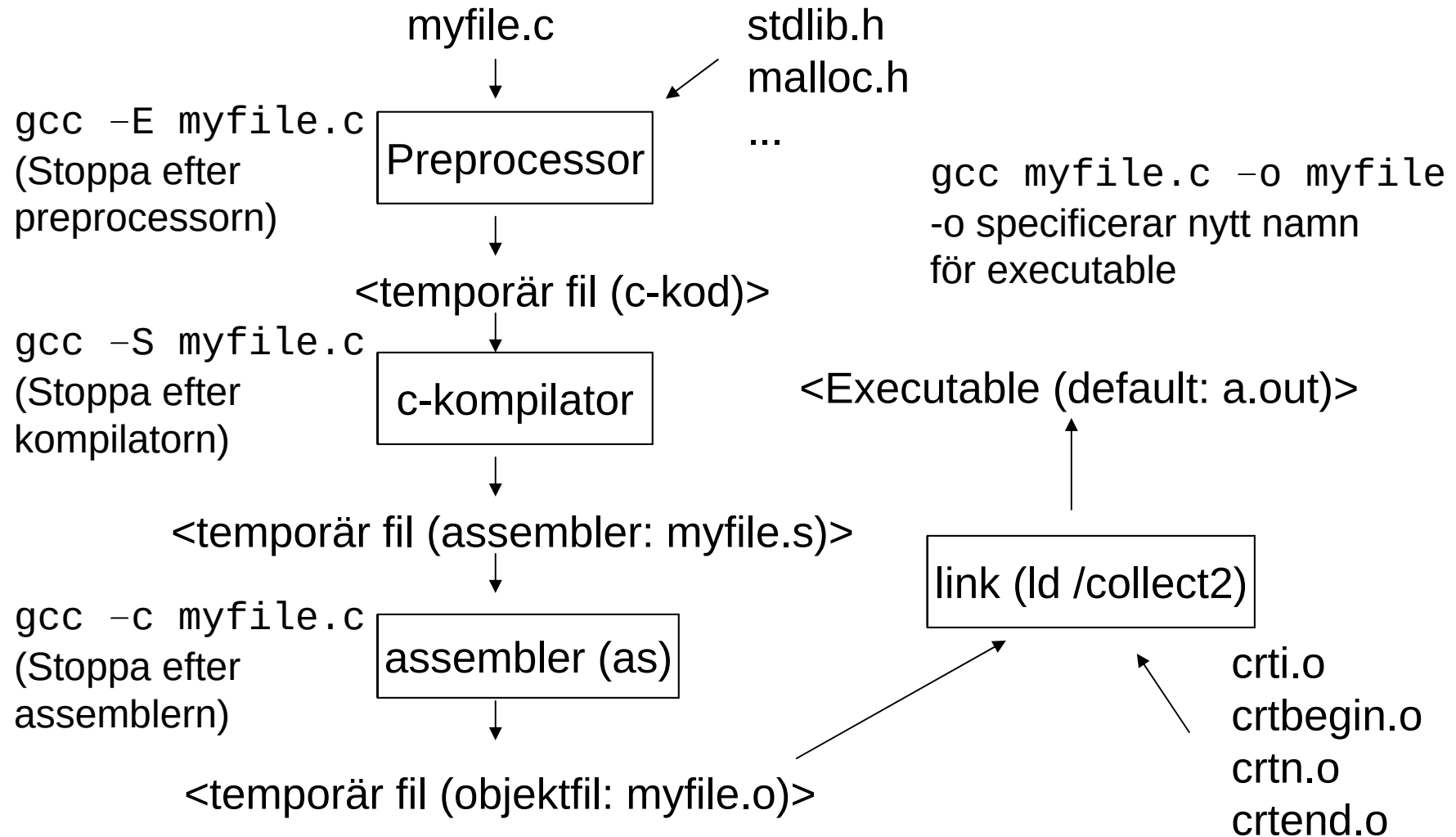
    printf("Max av 2 och 5 ar %i\n", (
        ((2)>(5))? (2) : (5) ));
    printf("Max av x och y ar %i\n", (
        ((x)>(y))? (x) : (y) ));

    printf("Debug: '%s\n", "Nu narmar sig
        slutet");
}

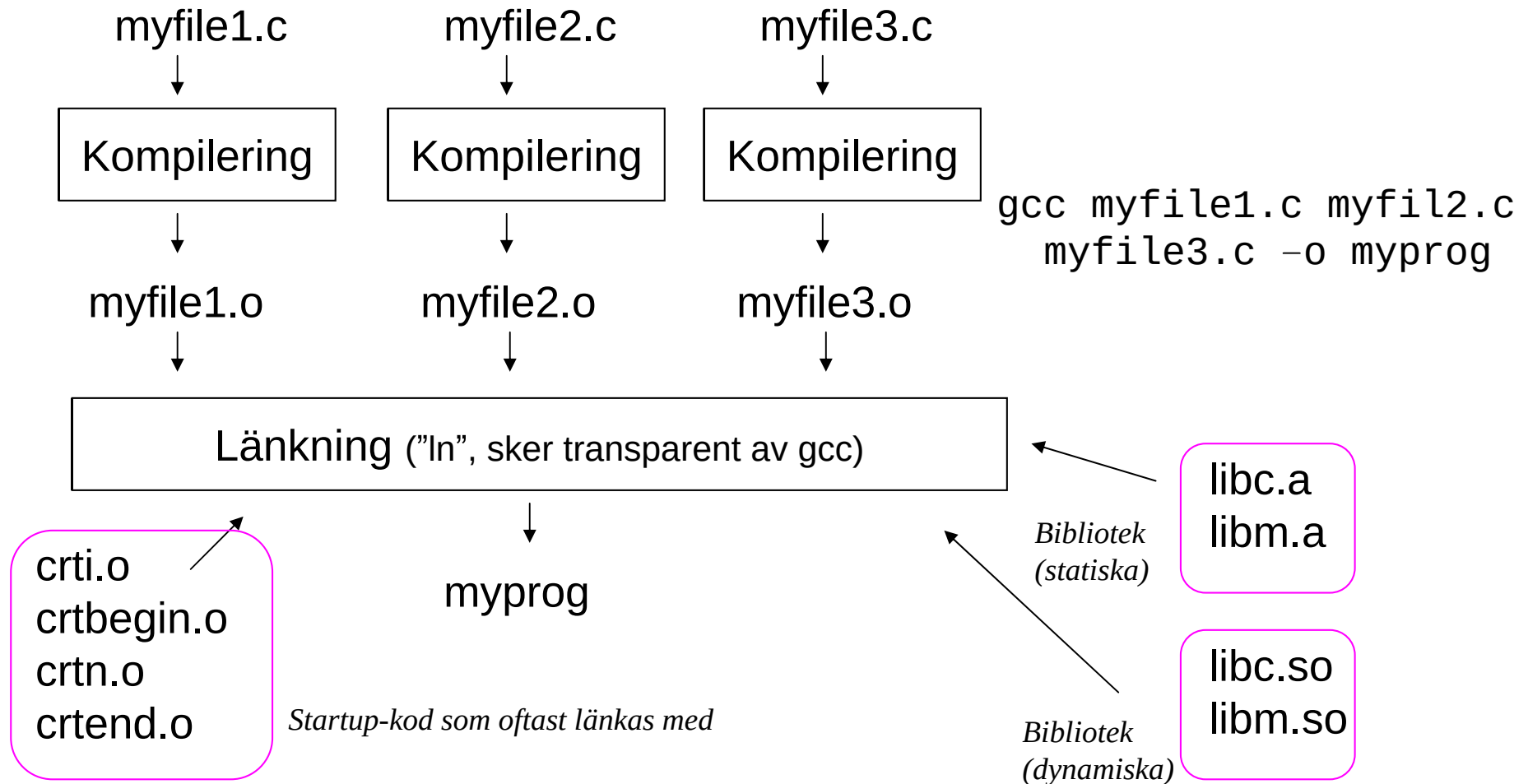
main () {
    int x = 33;
    int y = 12;

    printf("Max av 2 och 5 ar %i\n", (
        ((2)>(5))? (2) : (5) ));
    printf("Max av x och y ar %i\n", (
        ((x)>(y))? (x) : (y) ));
}
```

Kompileringsprocessen



Vanligen många källfiler



C++

Klasser i C++

```
#include <string>
class Person {
    std::string m_name;
    std::string m_address;
    int m_skonummer;
public:
    void setName(string const &str);
    void setAddress(string const &str);
    string const &name() const;
    string const &address() const;
};
```

Klassdeklaration = interface

Definition av funktioner i klassen = implementation

Klasser – konstruktor - destruktor

- Två specialfunktioner som har att göra med hur klassen internt fungerar – konstruktor och destruktor
 - Konstruktor – metod som efter att objektet skapats, men före access av detta
 - Samma namn som klassen, ingen returtyp, kan finnas överladdade konstruktörer

```
class Person {  
public:  
    Person();                /*Default konstruktor */  
    Person(string const &str); /* Överladdad */  
}  
main() {  
    Person p;                /* Default används */  
    Person p2("Axe1 Eklund"); /* Överladdad */  
}
```

Sammanstatta klasser

- Klasser har ofta i sin tur medlemmar som också är klasser → komposition
- Ex. Person-klassen har datafält av typen string, vilken i sin tur är en klass
- När vi har nästade klasser, hur kommer initialiseringen av de olika klasserna att gå till?
 - Kompilatorn skapar kod som initialiserar behövliga klassar, generar kod för anrop av behövliga konstruktorer

Referenser

- C++, som tillägg till variabler och pekare till variabler, "referenser" eller synonymer för variabler

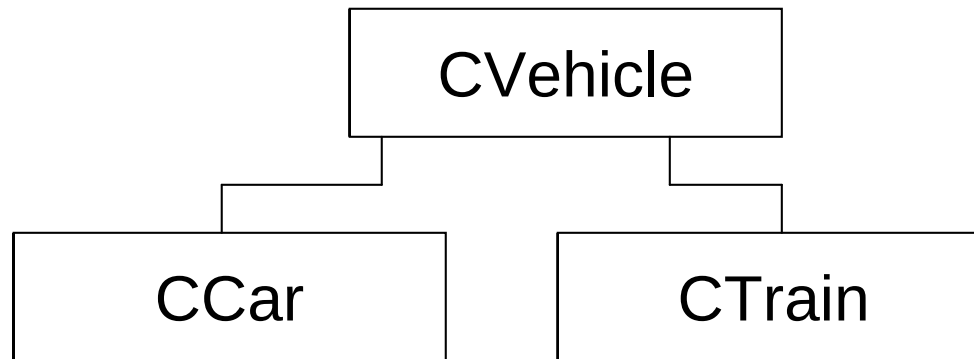
```
int value;  
int &ref=value; /* Referens */  
value++;        /* Alt 1 */  
ref++;          /* Alt 2 */
```

- Referenser möjliggör att överföra funktionsargument som kan modifieras (dvs. C:s "call by value" gäller inte mera)

Const i C++

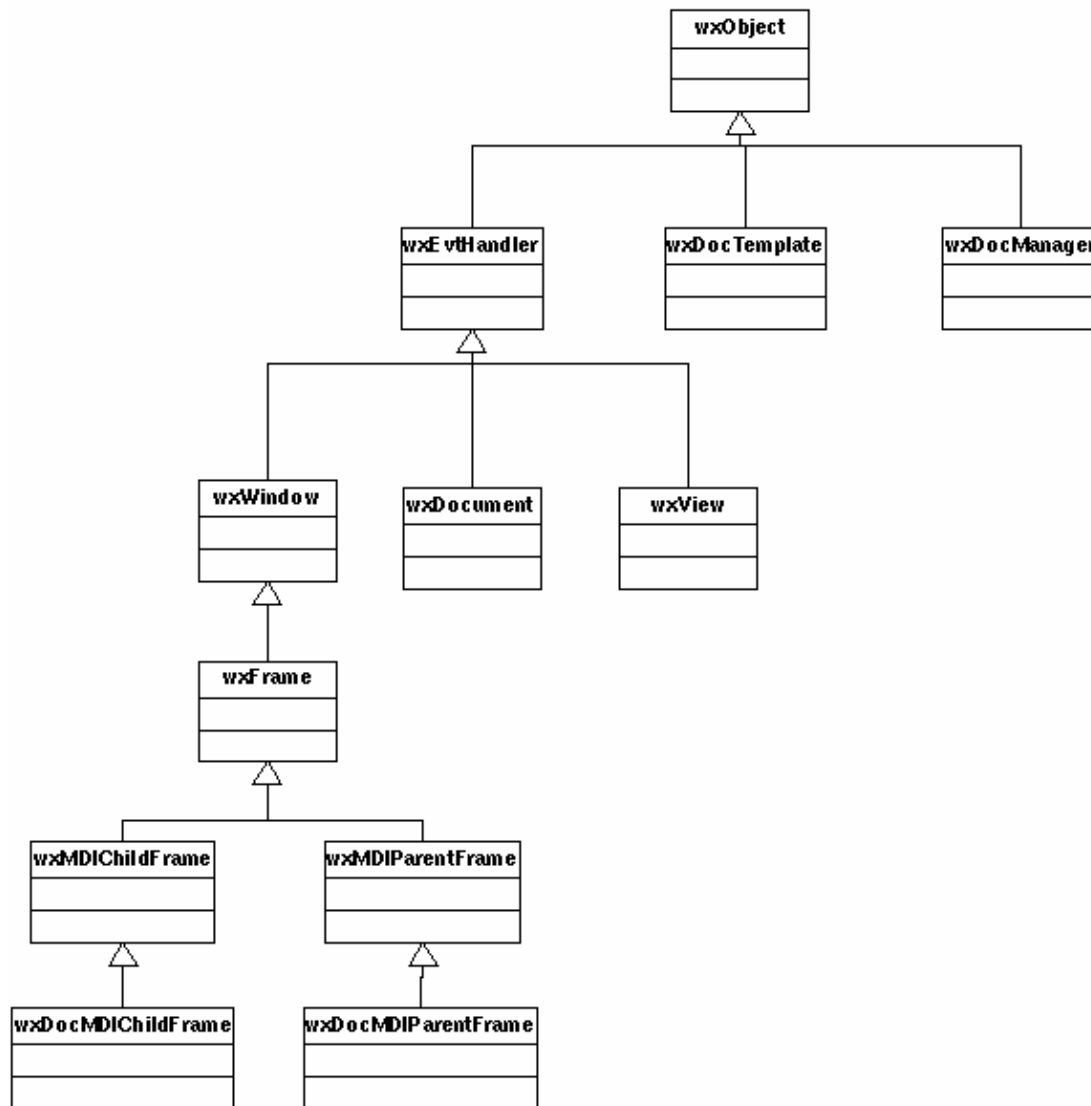
- Nyckelordet "const"
 - Även om "const" finns i C, används det ganska sällan. I C++ är det vanligt.
 - `char * const str; (1)`
 - `char const * str; (2)`
 - `const char * str; (3)`
 - Regel: Läs från variabelnamnet mot vänster
 - (1) Str är en konstant pekare till tecken
 - (2) Str är en pekare till konstant tecken
 - (3) Str är en pekare till tecken som är konstant
 - Notera. (2) och (3) är ekvivalenta
- ```
str = 'a'; / (1): OK, (2)(3): FEL! */
str++; /* (1): FEL!, (2)(3): OK */
```

# Ärvning



- **Dvs en CCar är en speciell typ av CVehicle**
- **Märk:**
  - **Privata (private:) klassmedlemmar är fortfarande privata, den ärvande klassen måste accessera dessa via funktioner**

# wxWidgets klassshierarki



# Polymorfism

- Normalt gäller att vi använder funktion enligt pekartyp
  - Men, C++ erbjuder en metod för att använda den deriverade klassens funktion trots att man använder sig av en pekare till basklassen → polymorfism
    - dvs. `vp->GetWeight()` ger 880 istället för 800 i exemplet på förra sidan
  - Genomförs via "late-binding", dvs länkning under exekvering, först då avgörs vilken funktion som verkligen skall anropas

# Polymorfism

- Polymorfism, genomförs via direktivet `virtual` för funktionsnamnet. Efter detta kommer den deriverade klassens motsvarande funktion att användas, även om en pekare till basklassen används.

```
class CVehicle {
public:
 CVehicle();
 CVehicle(float w);
 virtual float GetWeight();
private:
 float m_Weight;
}
```

- Notera, efter att en funktion i basklass har deklarerats *virtual*, kommer den att förbli det i alla deriverade klasser.

# Pure virtual functions

- Hittills har vi antagit att den virtuella funktionen också har en implementation i basklassen. Om vi inte har någon implementation i basklassen, talar man om *pure virtual functions*.
  - En pure virtual function skapas genom att till funktionsdeklarationen addera prefixet `virtual` och postfixet `=0`

```
class CVehicle {
 virtual float GetWeight() = 0;
}
```
- Vi har deklarerat ett *protokoll*, m.a.o. regler för vad som måste implementeras innan en deriverad klass kan användas

# Dynamiskt minnesallokering i C++

- Hittills har vi behandlat statiska eller automatiska klasser
- C++ inför *operatorerna* **new** och **delete** för att allokera dynamisk minne

```
main() {
 CVehicle *vp = new CVehicle(12.0);
 ... // Code
 delete vp; /* kallar även på destruktorn */
}
```

- Notera att **new** och **delete** också kallar på konstruktorn/destruktorn

# Överladdning av operatorer

- I C++ kan vi tilldela operatorer ny funktionalitet
  - t.ex. för assignment

```
class Person {
public:
 void operator=(Person const &other);
}
void Person::operator=(Person const &other) {
 delete m_namn;
 delete m_adress;

 m_namn = strdupnew(other.m_namn);
 m_adress = strdupnew(other.m_adress);
}
```



# Exceptioner

- Vid felsituationer i C, sker ofta något av följand
  - Funktionen noterar det ovanliga, och ger ett meddelande
  - Funktionen stoppar exekveringen och returnerar ett felmeddelande till kallaren
    - Den kallande funktionen kan i sin tur föra felmeddelanden uppåt i hierarkin
  - Funktionen gör beslutet att saker spårar ur och avslutar processen med t.ex. `exit()`
  - Funktionen använder en kombination av `setjmp()` och `longjmp()`, för att direkt återvända till en högre nivå

# Format på template-funktion

*Keyword*

*Kommaseparerad lista med templat-  
parameterlista*

**template** <typename Type>

```
Type add(Type const &lvalue, Type const
 &rvalue) {
 return lvalue + rvalue;
}
```

Vi kan notera, att hittills har vi endast använt formella variabelnamn som argument till funktioner, typerna har varit givna.

Nu låter vi även typerna på argumenten fungera som formella namn.

# Template-klasser

- Används i STL (Standard Template Library)
- Typisk för klasser som enkapsulerar andra datatyper såsom
  - Vektorer
  - Matriser
  - Länkade listor

# Komplexa klassen med templat

```
template <typename T1>
class Complex {
public:
 Complex(T1 re=0.0, T1 img=0.0) {m_re=re;m_img=img;}
 Complex(const Complex &other) {Copy(other);}

 Complex &operator=(const Complex &other) {Copy(other); return *this;}
 Complex operator*(const Complex &other) const;
 Complex operator/(const Complex &other) const;
 Complex operator+ (const Complex &other) const {Complex res(*this);
 res.m_re+=other.m_re;res.m_img+=other.m_img; return res;}
 //Complex const operator+(const Complex &other) {Complex res(*this);
 res.m_re+=other.m_re;res.m_img+=other.m_img; return res;}
 int a;
 //ostream &operator<<(ostream &stream, Complex const &c);

 T1 const Re() {return m_re;}
 T1 const Img() {return m_img;}
 const char *c_str();
private:
 void Copy(const Complex &other) {m_re=other.m_re; m_img=other.m_img;}
 double m_re;
 double m_img;
 char m_c_str[20];
};
```

# WxWidgets Hello World

- Skapa en egen klass som ärver wxApp, skriv en funktion OnInit() för denna klass

```
class HelloWorldApp : public wxApp {
public:
 bool OnInit();
};

DECLARE_APP(HelloWorldApp) // Skapar en global att nå denna appl.
IMPLEMENT_APP(HelloWorldApp) // Skapar kod, antingen main() eller WinMain()
 beroende på plattform

bool HelloWorldApp::OnInit() {
 wxFrame *frame = new wxFrame((wxFrame*) NULL, -1, "Hello World Title");
 // Skapar ett nytt fönster
 frame->CreateStatusBar(); // lägger till status bar
 frame->SetStatusText("Hello World Status Bar"); // Text i status-bar
 frame->Show(TRUE); // Visar fönstret
 SetTopWindow(frame); // Sätt till topp-fönster
 return true; // Om returnerar false, avslutas allt genast
}
```

# Makefile för Linux/Unix

```
// wxhello.mak: Usage: % make -f wxhello.mak
CXX = $(shell wx-config --cxx)
```

```
PROGRAM = wxhello #Obs, detta ändras åtminstone vid nytt prog.
```

```
OBJECTS = $(PROGRAM).o
```

```
implementation
```

```
.SUFFIXES: .o .cpp
```

```
.cpp.o :
 $(CXX) -c `wx-config --cxxflags` -o $@ $<
```

```
all: $(PROGRAM)
```

```
$(PROGRAM): $(OBJECTS)
 $(CXX) -o $(PROGRAM) $(OBJECTS) `wx-config --libs`
```

```
clean:
 rm -f *.o $(PROGRAM)
```

```

```

Notera användningen av `wx-config` (notera riktningen på `-tecknet  
Direkt även

```
% g++ `wx-config --cxxflags --libs` wxhello.cpp -o wxhello
```

# Addition av egenskaper

- Måste nu också ändra **OnInit**-funktionen tidigare, så att den istället instantierar ett objekt av klassen **MyFrame**

```
bool HelloWorldApp::OnInit() {
 MyFrame *frame = new MyFrame(NULL, "Hello World Title"); //
 Skapar ett nytt fönster
 frame->CreateStatusBar(); // lägger till status bar
 frame->SetStatusText("Hello World Status Bar"); // Text i
 status-bar
 frame->Show(TRUE); // Visar fönstret
 SetTopWindow(frame); // Sätt till topp-fönster
 return true; // Om returnerar false, avslutas allt genast
}
```

# Adderar meny

```
MyFrame::MyFrame(wxWindow *parent, const wxString& title)
: wxFrame(parent, -1, title)
{
 m_pTextCtrl = new wxTextCtrl(this, -1, wxString("Mata in text,
 tack"),wxDefaultPosition, wxDefaultSize, wxTE_MULTILINE);
 m_pMenuBar = new wxMenuBar();
 // File Menu
 m_pFileMenu = new wxMenu();
 // Notera, ampersand (&) ger automatisk keyboard-shortcut
 m_pFileMenu->Append(MENU_FILE_OPEN, "&Open");
 m_pFileMenu->Append(MENU_FILE_SAVE, "&Save");
 m_pFileMenu->AppendSeparator();
 m_pFileMenu->Append(MENU_FILE_QUIT, "&Quit");
 m_pMenuBar->Append(m_pFileMenu, "&File");
 // About menu
 m_pInfoMenu = new wxMenu();
 m_pInfoMenu->Append(MENU_INFO_ABOUT, "&About");
 m_pMenuBar->Append(m_pInfoMenu, "&Info");
 SetMenuBar(m_pMenuBar); //Här sätter vi ny menyrad till fönstret
}
```



# Koppla aktioner till menyn

- Läger till klass-definition för **MyFrame** följande metoder, som ju beskriver vad vi vill göra

```
class MyFrame : wxFrame {

public:
 void OnMenuFileOpen(wxCommandEvent &event);
 void OnMenuFileSave(wxCommandEvent &event);
 void OnMenuFileQuit(wxCommandEvent &event);
 void OnMenuInfoAbout(wxCommandEvent &event);

protected:
 DECLARE_EVENT_TABLE() //Klassen har en event table

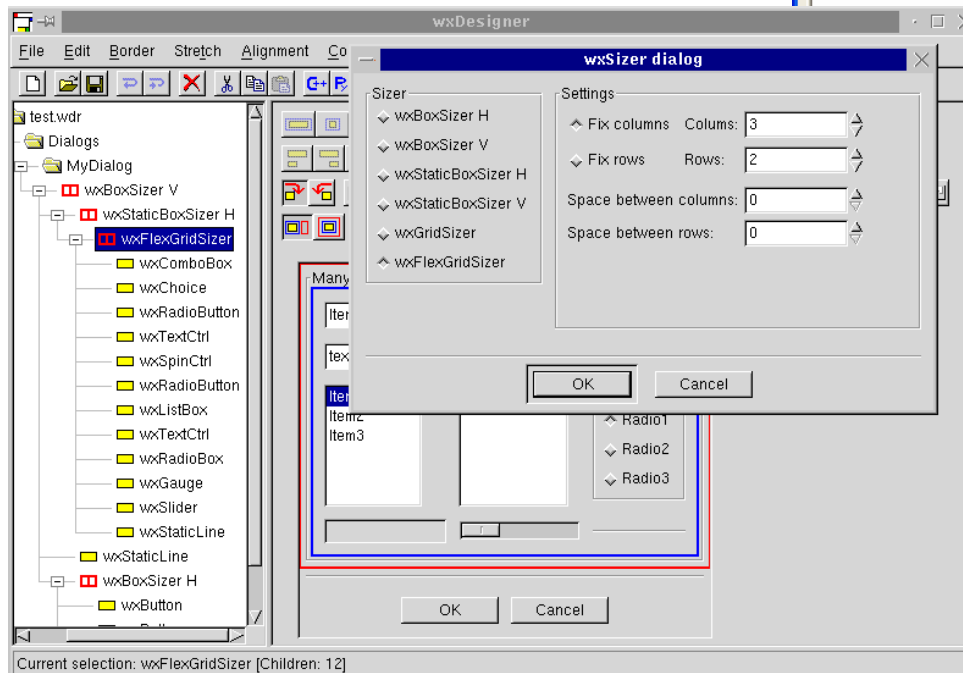
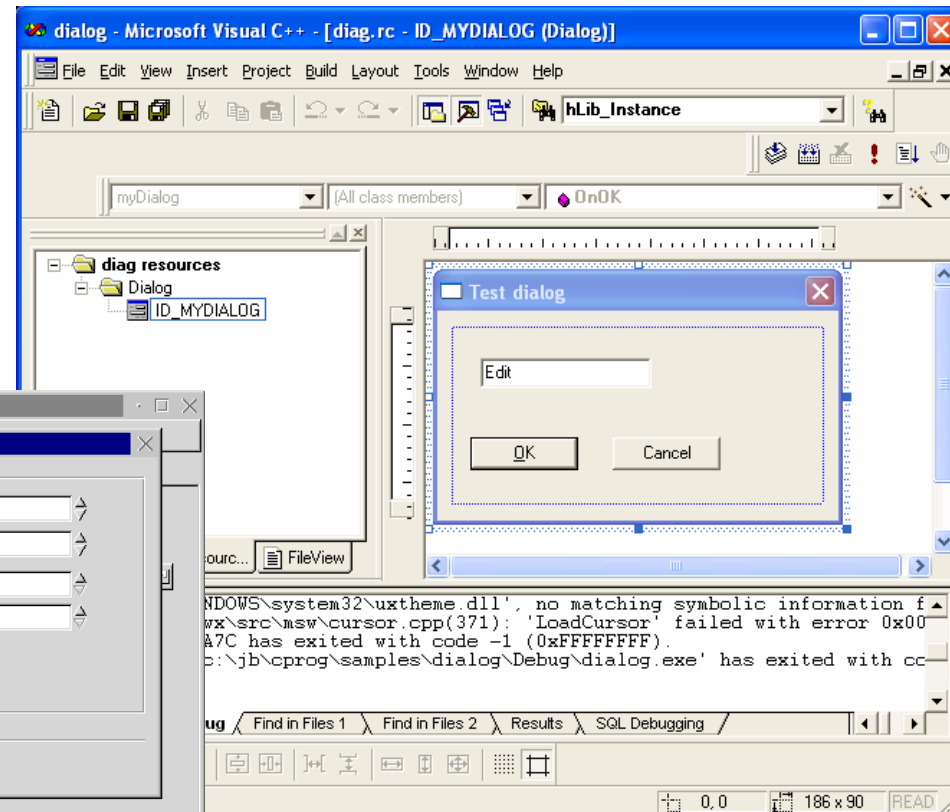
}
```

# Device Context - DC

- En Device Context (DC) är en generaliserad modell av ett område man kan rita på
  - Linjer, text, cirklar, färger etc.etc.
- När man ritar, gör man det alltid på en DC, inte t.ex. direkt på fönstret
- En DC kan också representera
  - bitkarta, printer, ..., bara man tar i beaktande skalningen
- I wxWidgets representeras denna DC av grundklassen wxDC

# I praktiken används ofta resurs-editorer

Visual studio



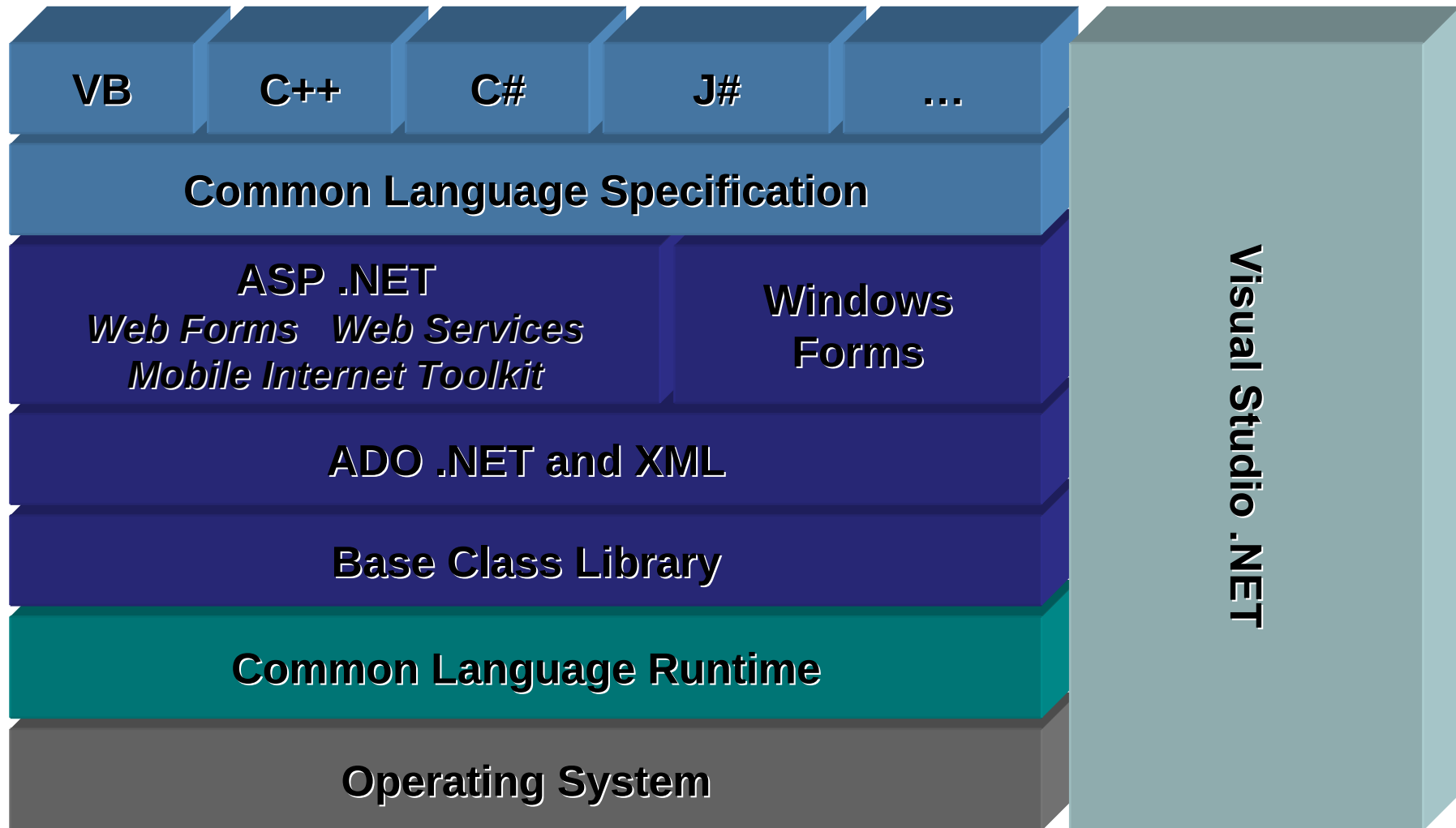
wxDesigner

# COM utveckling

- Microsoft office tools for creating documents
  - Print, cut, copy, paste
  - Local software bus to link office tools
  - Dynamic Data Exchange (DDE) - Internal standard, slow, error prone
- Object linking and embedding (OLE)
  - How to create compound documents
  - Using DDE (first)
- Compound Object Model (COM)
  - Local software bus to link general processing components
  - Replaces DDE
- ActiveX of OLE
  - Uses COM (instead of DDE) to define compound documents
  - Provides services
    - Name & property services, event based communication service
    - Document processing specific services (cut, copy, paste, ...)
- Distributed COM (DCOM)
  - Based on Distributed Computing Environment (DCE) + COM
  - Software bus with remote transparency

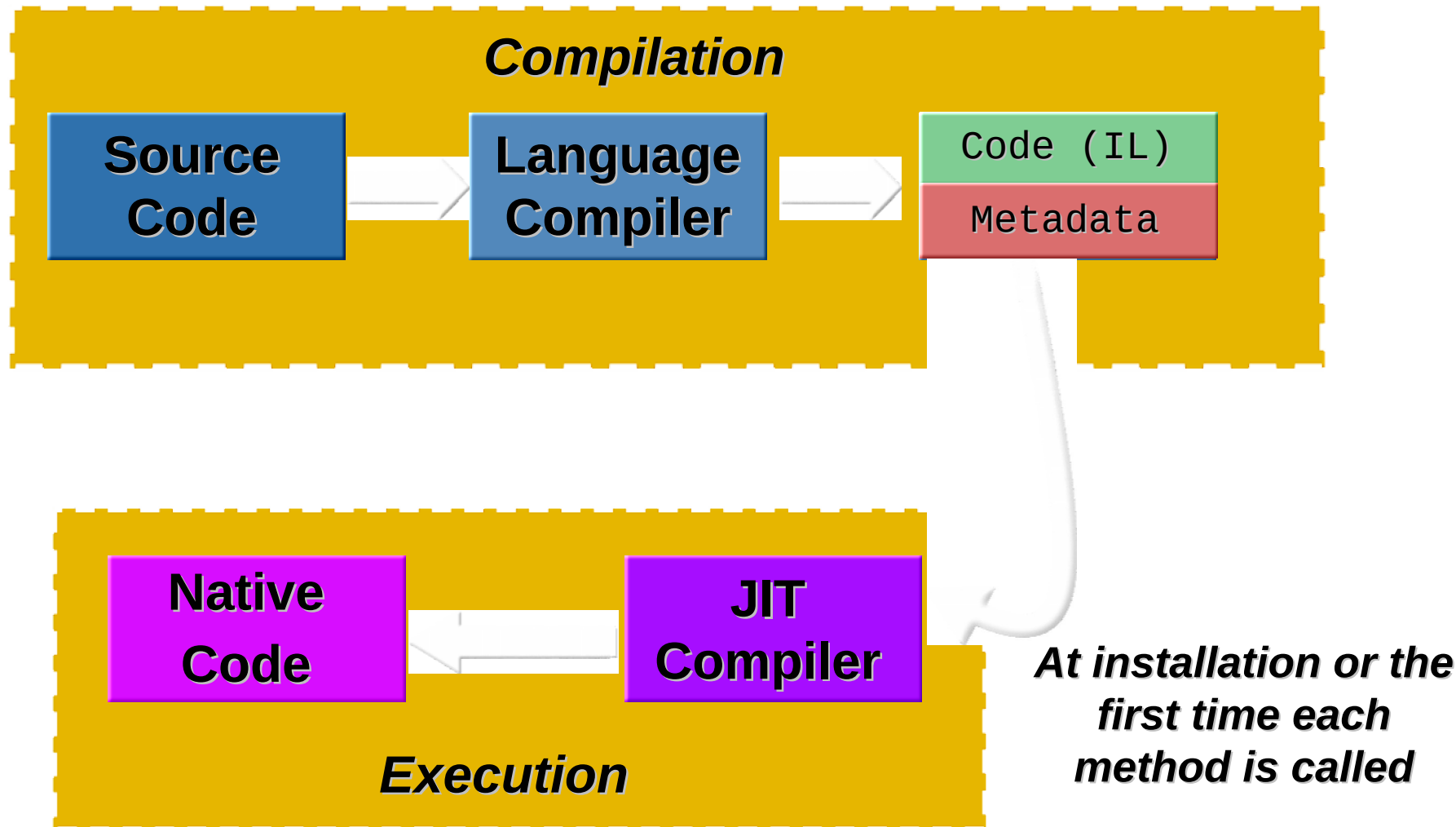
# Implementation and Benefits

.NET Framework and Tools



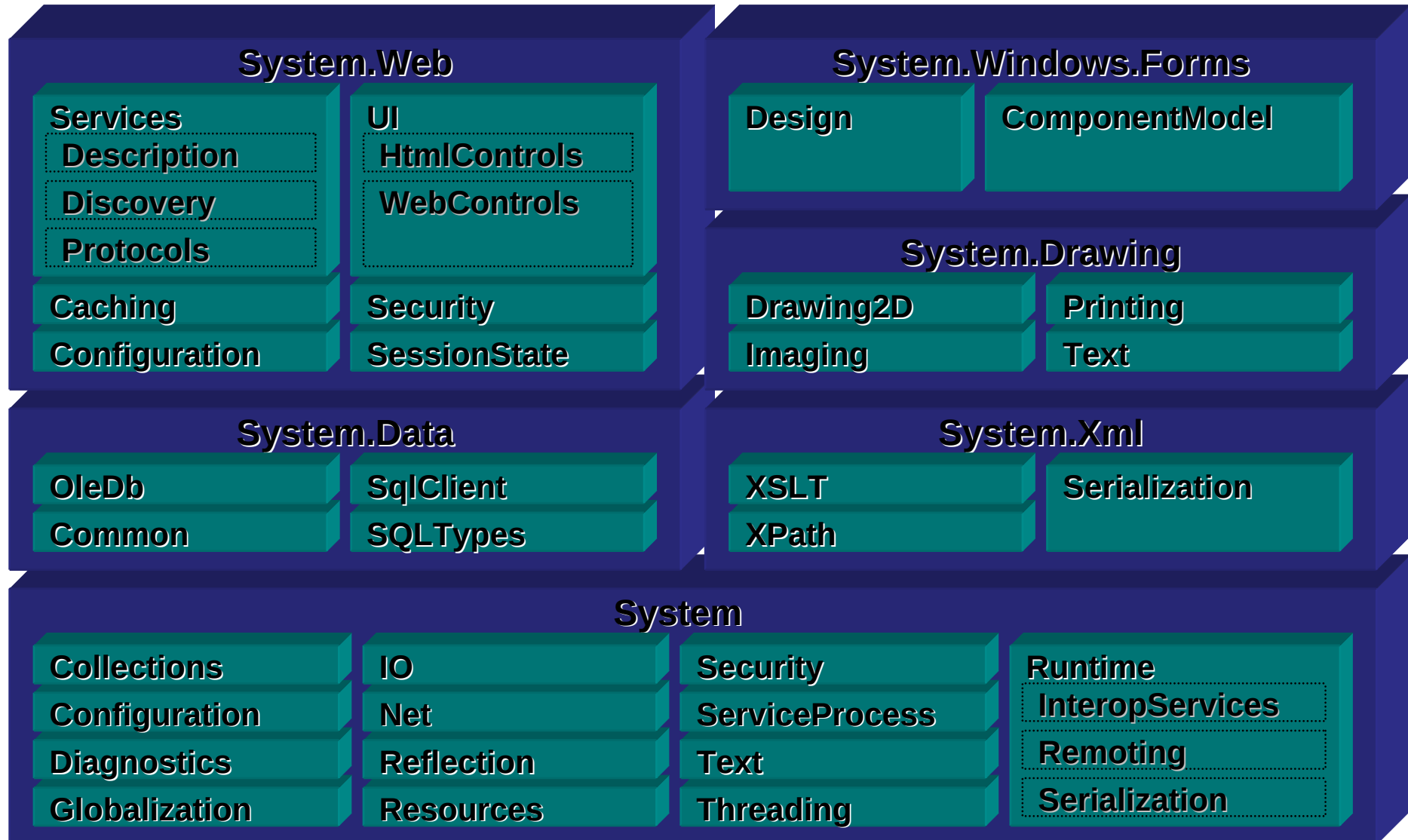
# Implementation and Benefits

## Compilation and Execution



# Implementation and Benefits

## .NET Framework Class Library



This document was created with Win2PDF available at <http://www.win2pdf.com>.  
The unregistered version of Win2PDF is for evaluation or non-commercial use only.